



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

Submitted by:

Jannatul Ferdoushi Islam

Student ID: 23101119

Section: C-1

Semester: 4-1

Submitted to:

Atia Rahman Orthi,

Lecturer,

University of Asia Pacific

Lab Report: FCFS CPU Scheduling

Problem Statement

Using the FCFS scheduling method, determine the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average Turnaround Time and Average Waiting Time for the given processes. The order in which the processes execute should also be shown.

Objective

The purpose of this experiment is to learn how the First Come First Serve (FCFS) CPU scheduling technique works. It also focuses on finding CT, TAT and WT for each process and calculating the average turnaround time, average waiting time and the process execution order.

Given Data

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	3	5
P2	2	4
P3	4	3
P4	1	2
P5	5	3

Working Principle

FCFS follows a simple rule: the process that reaches the ready queue earlier is served first. Therefore, the processes are scheduled according to arrival time. After finding the completion time from the execution sequence, Turnaround Time is obtained using $TAT = CT - AT$, and Waiting Time is obtained using $WT = TAT - BT$.

Source Code

```
processes = [  
    ["P1", 3, 5],  
    ["P2", 2, 4],  
    ["P3", 4, 3],  
    ["P4", 1, 2],  
    ["P5", 5, 3]  
]
```

```

order = sorted(processes,
key=lambda x: x[1])

result = {}
current_time = 0
execution = []

for p in order:
    pid = p[0]
    at = p[1]
    bt = p[2]

    if current_time < at:
        current_time = at

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt

    current_time = ct

    result[pid] = [ct, tat, wt]
    execution.append(pid)

print("P_ID   AT   BT   CT   TAT
WT")

for p in processes:
    pid = p[0]
    at = p[1]
    bt = p[2]

    ct = result[pid][0]
    tat = result[pid][1]
    wt = result[pid][2]

```

```
print(f"{pid:<7}{at:<5}{bt:<5}{ct:<5}{tat:<6}{wt}")
```

```
total_tat = 0
```

```
total_wt = 0
```

```
for p in processes:
```

```
    total_tat += result[p[0]][1]
```

```
    total_wt += result[p[0]][2]
```

```
avg_tat = total_tat /  
len(processes)
```

```
avg_wt = total_wt / len(processes)
```

```
print()
```

```
print("AVG TAT =", avg_tat)
```

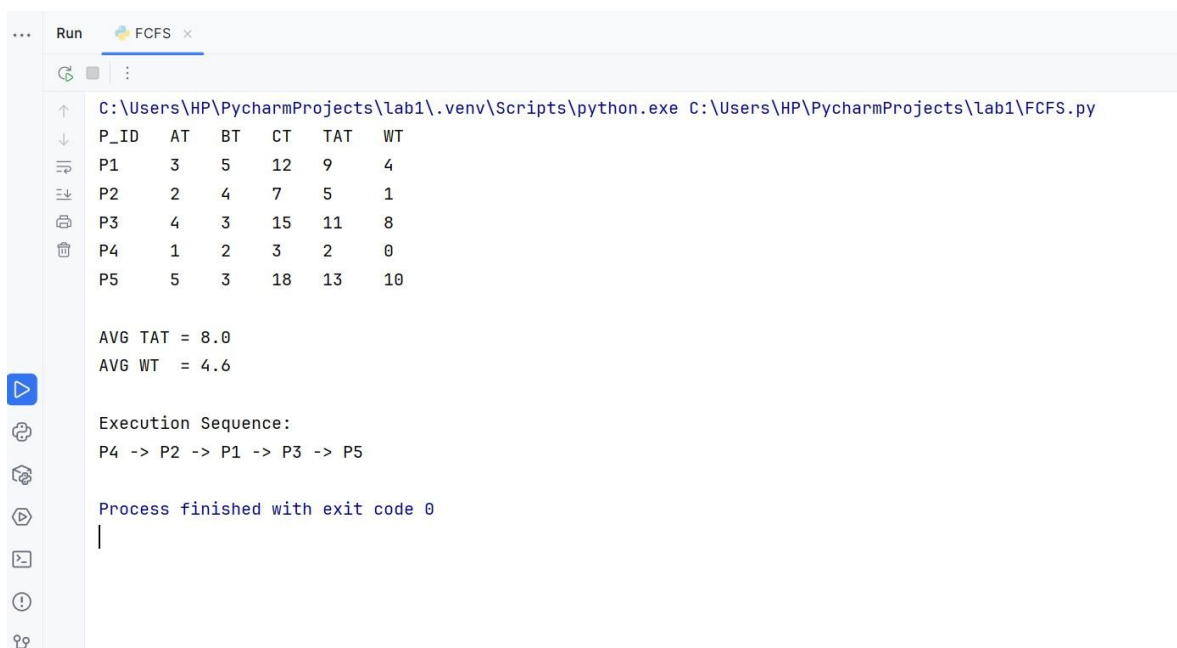
```
print("AVG WT  =", avg_wt)
```

```
print()
```

```
print("Execution Sequence:")
```

```
print(" -> ".join(execution))
```

Output



```
... Run FCFS x
C:\Users\HP\PycharmProjects\lab1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\lab1\FCFS.py
P_ID  AT  BT  CT  TAT  WT
P1    3  5  12  9    4
P2    2  4  7   5    1
P3    4  3  15  11    8
P4    1  2  3   2    0
P5    5  3  18  13   10

AVG TAT = 8.0
AVG WT  = 4.6

Execution Sequence:
P4 -> P2 -> P1 -> P3 -> P5

Process finished with exit code 0
|
```

Discussion

In this experiment, FCFS scheduled the processes based on their arrival times. The resulting execution sequence was P4, P2, P1, P3 and P5. CT, TAT and WT were found for every process, while the calculated average TAT was 8.0 and the average WT was 4.6. The result shows how arrival time determines the processing order and influences how long a process has to wait.

Conclusion

The FCFS CPU scheduling algorithm was successfully implemented using Python. All required values, including CT, TAT, WT, their averages and the execution sequence, were produced correctly. The experiment demonstrated the basic behavior of FCFS, where processes are handled in arrival order and each process waits for the earlier process to complete.

Github Link: